❹ Builds a User object using data from OidcUser (extracted from the ID Token)

In the next chapter, which focuses on authorization strategies, we'll configure Keycloak to include a custom `roles` claim in the ID Token and use that value to build the `User` object in the `UserController` class. Until then, we'll use a fixed list of values.

For Spring Web MVC and WebFlux controllers, besides using `ReactiveSecurityContextHolder` directly, we can use the annotations `@CurrentSecurityContext` and `@AuthenticationPrincipal` to inject the `SecurityContext` and the principal (in this case, `OidcUser`) respectively.

Let's simplify the implementation of the `getUser()` method by injecting the `OidcUser` object directly as an argument. The final result for the `UserController` class is shown in the following listing.

Listing 11.9 Returning information about the currently authenticated user

```
package com.polarbookshop.edgeservice.user;

import java.util.List;
import reactor.core.publisher.Mono;
import org.springframework.security.core.annotation.
➥AuthenticationPrincipal;
import org.springframework.security.oauth2.core.oidc.user.OidcUser;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class UserController {

  @GetMapping("user")
  public Mono<User> getUser(
   @AuthenticationPrincipal OidcUser oidcUser     ❶
  ) {
    var user = new User(                          ❷
```